Avatar Upload and Crop

Problem: Let a user pick a profile picture, crop it to a square with drag and zoom, preview it as a circle, and upload the result.

Tech stack: React (Vite, JavaScript) *(default, not specified)*

Time limit: 60 minutes *(default, not specified)*

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what must it do, what is out of scope.
2. **List interactions**: every user action and the expected result. This becomes the checklist.
3. **Sketch components**: who renders what, who owns which state.
4. **Design state and data**: the smallest state that describes the UI; derive the rest.
5. **Define contracts**: props, function signatures, API payloads (mock if no API).
6. **Find the hard parts early**: the 2-4 things that can sink you; plan them before coding.
7. **Build in layers**: the simplest working version first, then add one working layer at a time.
8. **Handle states and edge cases**: loading, empty, error, invalid input, races.
9. **Wrap up**: explain trade-offs and what you would add with more time.

Time budget (60 minutes)

Time	Activity
0-7 min	Requirements, clarifying questions, assumptions
7-12 min	Components, state, crop math on paper
12-22 min	File pick + validation + object URL preview
22-40 min	Crop viewport: cover-fit, pan (pointer), zoom (slider), clamping
40-48 min	Canvas export to Blob, mock upload, upload states
48-55 min	Keyboard, wheel, drag and drop, error states, cleanup
55-60 min	Walk through edge cases and trade-offs

Think out loud

Say what you are doing and why before you type: "*I'll keep the zoom and pan in one state object so a wheel event and a drag never read a stale value.*" The interviewer grades your reasoning as much as your code. If you get stuck, say what you are trying, so they can nudge you.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Pick an image by **file input** or **drag and drop**.
- **Validate** type (JPG, PNG, WebP) and size (max 5 MB) before loading.
- Show the image in a **square crop window** with a **circular guide**.
- **Pan** the image by dragging (mouse and touch), **zoom** with a slider (and mouse wheel).
- The image must **always cover** the crop window: no empty corners.
- **Save** exports the visible square as a small image (256 x 256) and **uploads** it.
- Show a **circular preview** with the upload status: uploading, saved, failed with **retry**.
- **Cancel** returns to the picker without changing the current avatar.

Non-functional requirements

Area	What it means here
Performance	Pan/zoom with CSS `transform` only (no re-layout, no canvas redraw per frame). Export a small 256 px image, not the 12 MP original.
Memory	Object URLs keep the whole file alive; revoke them when replaced or unmounted.
Error handling	Invalid type, too large, corrupt image, export failure, upload failure all show a clear message.
Accessibility	Keyboard pan (arrows) and zoom (+ / -), labelled slider, focus ring, `role="alert"` for errors, `aria-live` for upload status.
Usability	Works with touch (`touch-action: none`), wheel does not scroll the page, same file can be picked twice.
Reusability	Crop math is pure functions; `AvatarCropper` takes `src` and returns a `Blob` , so it can be reused for cover photos.
Extensibility	Output size, viewport size, max zoom are constants; the upload is one service function to swap for `fetch` .

Clarifying questions

1. Should the saved file be a **circle** (transparent corners) or a **square** shown as a circle with CSS? *(Most products store a square.)*
2. What **output size and format** does the backend expect? 256 px WebP/PNG, or the full-resolution crop?
3. What are the **file limits**: types, max size, minimum dimensions?
4. Do we need **rotation / flip**, or only pan and zoom?
5. Upload **immediately on save**, or a separate "Upload" step? Optimistic preview OK?
6. Is there a **real API** (endpoint, auth, response shape) or should I mock it?

Assumptions we build with

- Square crop, shown as a circle with CSS; output **256 x 256**, WebP (PNG fallback).
- JPG / PNG / WebP, **max 5 MB**.
- Pan + zoom only (zoom 1x-3x). No rotation.
- **Save = crop + upload**, with an optimistic preview and a retry on failure.
- Upload is **mocked** with a 1 s delay and a 20 % failure rate.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Click the drop zone, choose a valid image	Cropper opens with the image covering the window, zoom = 1, centered
Drop an image file on the drop zone	Same as above; zone highlights while dragging over
Pick a `.txt` or `.gif`	Error: "Please choose a JPG, PNG or WebP image." Cropper does not open
Pick a 6 MB image	Error: "Image must be under 5 MB."
Pick a file named `.png` that is not a real image	Error: "Could not read this image." Back to drop zone
Drag the image	Image follows the pointer; stops at the edge (never shows empty space)
Move the zoom slider / scroll the wheel	Image scales around the window center; offset re-clamped; page does not scroll
Focus the crop window, press arrows	Image moves 10 px per press, clamped
Press `+` / `-`	Zoom in / out by 0.1, clamped to 1-3
Click Save	Crop exported to 256 px Blob, preview updates, status "Uploading..."
Upload succeeds	Status "Avatar saved"
Upload fails	Status "Upload failed." + Retry button
Click Retry	Same Blob uploaded again
Click Cancel	Back to drop zone; current avatar unchanged
Pick a new image while the previous upload is in flight	Only the latest upload's result updates the status

This table is the implementation and test checklist.

---

3. Design the architecture

Module	Responsibility
`App`	Owns the flow: picked source, cropped avatar, upload status, error.
`FileDropzone`	File input + drag and drop; hands the raw `File` up.
`AvatarCropper`	Shows the image in the crop window; pan, zoom, keyboard, wheel; exports a `Blob`.
`AvatarPreview`	Circular avatar + upload status + Retry.
`utils/cropMath.js`	Pure math: cover scale, clamping, zoom around center, crop rectangle.
`utils/image.js`	`validateFile`, `cropImage` (canvas to Blob).
`services/avatarApi.js`	`uploadAvatar(blob)` (mock today, `fetch` tomorrow).

```
App  (source, avatar, status, error)
├── AvatarPreview      ← url, status, onRetry
├── FileDropzone       ← onFile            (when no source)
│     └── validateFile()
└── AvatarCropper      ← src, onCrop, onCancel, onError   (when source)
      ├── cropMath: getMinScale / panView / zoomView / getCropRect
      └── image.cropImage() → Blob → App.handleCrop → uploadAvatar()
```

Start simple: one `App` with an `<input type="file">` and an `<img>`. Split out `AvatarCropper` once pan/zoom state appears, and pull the math into pure functions as soon as it gets tricky, so it can be tested without a DOM.

---

4. Design the state / data model

State or field	Owner	Purpose
`source`	App	Object URL of the picked file; `null` = show drop zone, set = show cropper
`avatar`	App	`{ blob, url }` of the cropped result; blob kept for Retry
`status`	App	`'idle' | 'uploading' | 'success' | 'error'`
`error`	App	Validation / read / export message
`requestRef`	App (ref)	Id of the latest upload, ignores stale responses
`img`	AvatarCropper	Natural size `{ w, h }` once loaded; `null` = still loading
`view`	AvatarCropper	`{ zoom, x, y }`: zoom 1-3, offset of image center from window center (px)
`saving`	AvatarCropper	Disables buttons while exporting
`dragRef`	AvatarCropper (ref)	Pointer start position during a drag

```
// App
const [source, setSource] = useState(null);
```

```
const [avatar, setAvatar] = useState(null);   // { blob, url }
const [status, setStatus] = useState('idle'); // idle | uploading | success | error
const [error, setError] = useState('');
const requestRef = useRef(0);

// AvatarCropper
const [img, setImg] = useState(null);                       // { w, h }
const [view, setView] = useState({ zoom: 1, x: 0, y: 0 }); // one object = one atomic update
```

Senior note:

- **Derived, not stored:** the real scale (`getMinScale(img) * zoom`), the displayed image size, the CSS transform and the crop rectangle are all computed from `img` + `view` . Storing them would let them drift apart.
- `zoom` **and offset live together** in `view` , because zooming changes the offset. Two separate states would need two updates that can read stale values.
- **Refs for things that don't render:** drag start point and upload id change often but never change the UI.
- **Ownership:** the cropper owns transient editing state and is reset with `key={source}` when a new image is picked; `App` owns what survives the cropper (avatar, upload status).

---

5. Design the data structures and contracts

Example data

```
const img  = { w: 1000, h: 500 };          // natural size from <img>.naturalWidth/Height
const view = { zoom: 1.5, x: -280, y: 0 }; // CSS px, relative to the window center
// VIEWPORT = 280 → minScale = max(280/1000, 280/500) = 0.56
// displayed size at zoom 1.5 = 1000*0.84 × 500*0.84 = 840 × 420
// allowed x range = ±(840-280)/2 = ±280, y range = ±(420-280)/2 = ±70
```

The offset is stored **relative to the center**, not as top/left. That makes "centered" equal to `{x: 0, y: 0}` , the clamp symmetric (`±max`), and zoom around the center a simple multiplication. The object URL string doubles as a **stable id** for the cropper (`key={source}`).

Contracts

Function / component	Input	Output
`validateFile(file)`	`File`	`null` or error string
`getMinScale(img, v)`	`{w,h}` , viewport px	scale so the image covers the viewport
`panView(view, x, y, img)`	wanted offset	new clamped `view`
`zoomView(view, zoom, img)`	wanted zoom	new `view` with clamped zoom and offset
`getCropRect(view, img)`	current view	`{ sx, sy, size }` in natural image pixels
`cropImage(imgEl, rect)`	`<img>` , rect	`Promise<Blob>` (256 x 256)

Function / component	Input	Output
`uploadAvatar(blob)`	`Blob`	`Promise<{ id, size, uploadedAt }>`, rejects on failure
`<AvatarCropper src onCrop onCancel onError />`	object URL	`onCrop(blob)` / `onError(message)`

Mock API (swap for a real endpoint):

```
POST /api/me/avatar        Content-Type: multipart/form-data   (field "avatar")
200 { "id": "8f2c...", "size": 18342, "uploadedAt": 1767312000000 }
4xx/5xx → show "Upload failed." + Retry
```

6. Trace the data flow

Key interaction: **pick image → zoom → drag → Save**.

```
User picks photo.jpg (2.1 MB)
   |
   ▼
FileDropzone onChange → onFile(file)
   |
   ▼
App.handleFile: validateFile(file) → null (valid)
   setSource(URL.createObjectURL(file))          ── state: source = "blob:..."
   |
   ▼
<AvatarCropper key={source}> mounts, <img src> starts loading
   |  img.onLoad → setImg({ w: 4000, h: 3000 })
   ▼
render: minScale = max(280/4000, 280/3000) = 0.0933
        img style width 373px, height 280px, translate(-50%,-50%) scale(1)
   |
User moves the slider to 2
   |  setView(v => zoomView(v, 2, img))  → { zoom: 2, x: 0, y: 0 }
   ▼
User drags 60px right
   |  pointermove → setView(v => panView(v, 0 + 60, 0, img))
   |  clamp: maxX = (4000*0.1867 - 280)/2 = 233 → x = 60
   ▼
User clicks Save
   |  getCropRect(view, img) → { sx: 928.6, sy: 750, size: 1500 }
   |  cropImage(imgEl, rect) → canvas.drawImage → toBlob('image/webp')
   ▼
App.handleCrop(blob)
   setAvatar({ blob, url })  setSource(null)      ── preview shows new avatar (optimistic)
   upload(blob): id = ++requestRef.current; setStatus('uploading')
   |
   ▼
uploadAvatar resolves → id === requestRef.current → setStatus('success')
   |
```

```
   ▼
UI: circular preview + "Avatar saved"; old object URLs revoked by effect cleanups
```

7. Important technical challenges

Challenge 1: Mapping what the user sees back to real pixels

Problem: The user sees a 4000 x 3000 photo shrunk to 373 x 280 px, zoomed 2x and moved 60 px. `canvas.drawImage` needs the source rectangle in **natural** pixels. Getting this wrong saves a slightly shifted or stretched avatar, which is the bug interviewers look for.

Solution: Keep one model: `scale = minScale * zoom`, offset = image center relative to window center. Then the visible square in natural pixels has side `V / scale`, and its top-left is the image center minus half that side minus the offset converted back to natural pixels.

```
export function getCropRect(view, img, v = VIEWPORT) {
  const scale = getMinScale(img, v) * view.zoom;
  const size = v / scale;
  return {
    sx: img.w / 2 - size / 2 - view.x / scale,
    sy: img.h / 2 - size / 2 - view.y / scale,
    size,
  };
}
// canvas: ctx.drawImage(imgEl, sx, sy, size, size, 0, 0, 256, 256)
```

Challenge 2: The image must always cover the crop window

Problem: A 1000 x 500 photo at "fit" scale leaves black bars; after zooming in and dragging to the corner, zooming back out leaves an empty corner. The exported avatar would have a black stripe.

Solution: (a) Zoom 1 = **cover** scale: `max(V/w, V/h)`, so the short side fills the window. (b) After every pan **and** every zoom, clamp the offset to `±(displayed - V) / 2`. (c) Zoom around the center by multiplying the offset by the zoom ratio, then clamp.

```
export const getMinScale = (img, v = VIEWPORT) => Math.max(v / img.w, v / img.h);

export function clampOffset(x, y, img, scale, v = VIEWPORT) {
  const maxX = (img.w * scale - v) / 2;
  const maxY = (img.h * scale - v) / 2;
  return { x: clamp(x, -maxX, maxX), y: clamp(y, -maxY, maxY) };
}

export function zoomView(view, nextZoom, img, v = VIEWPORT) {
  const zoom = clamp(nextZoom, 1, MAX_ZOOM);
  const ratio = zoom / view.zoom;
  return { zoom, ...clampOffset(view.x * ratio, view.y * ratio, img, getMinScale(img, v) * zoom, v) };
}
```

Challenge 3: Smooth pan / zoom on mouse, touch, wheel and keyboard

Problem: Mouse events don't cover touch. If the pointer leaves the window mid-drag the drag gets stuck. React's `onWheel` is passive, so `preventDefault` can't stop the page from scrolling. Wheel events fire many times per frame; reading `zoom` from the closure gives stale values.

Solution: Pointer events + `setPointerCapture` (one code path for mouse, pen and touch; keeps getting moves outside the box) and `touch-action: none`. A native wheel listener with `{ passive: false }`. **Functional updates** on one `view` object so every event builds on the latest state. Move the image with CSS `transform` only (GPU, no layout).

```
const handlePointerDown = (e) => {
  e.currentTarget.setPointerCapture(e.pointerId);
  dragRef.current = { px: e.clientX, py: e.clientY, x: view.x, y: view.y };
};
const handlePointerMove = (e) => {
  const d = dragRef.current;
  if (!d) return;
  setView((v) => panView(v, d.x + e.clientX - d.px, d.y + e.clientY - d.py, img));
};

useEffect(() => {
  const el = viewportRef.current;
  if (!img) return;
  const onWheel = (e) => {
    e.preventDefault();
    setView((v) => zoomView(v, v.zoom - e.deltaY * 0.002, img));
  };
  el.addEventListener('wheel', onWheel, { passive: false });
  return () => el.removeEventListener('wheel', onWheel);
}, [img]);
```

Challenge 4: Memory leaks and stale uploads

Problem: `URL.createObjectURL` keeps a 5 MB file in memory until revoked; a user trying 10 photos leaks 50 MB. And if upload #1 is slow and fails *after* upload #2 succeeded, the UI would wrongly show "Upload failed".

Solution: Revoke in an effect cleanup keyed on the URL (runs when it changes or on unmount). Tag each upload with an increasing id and only apply the result if it is still the latest.

```
useEffect(() => () => source && URL.revokeObjectURL(source), [source]);
useEffect(() => () => avatar && URL.revokeObjectURL(avatar.url), [avatar]);

const upload = async (blob) => {
  const id = ++requestRef.current;
  setStatus('uploading');
  try {
    await uploadAvatar(blob);
    if (id === requestRef.current) setStatus('success');
  } catch {
    if (id === requestRef.current) setStatus('error');
  }
};
```

8. Implement in the correct order

#	Step	Working result
1	`<input type="file">` + `URL.createObjectURL` → `<img>`	Picked image shows on screen
2	`validateFile` + error message	Wrong type / too big is rejected
3	Crop window (fixed 280 px, `overflow: hidden`) + cover scale on `onLoad`	Image fills the square, centered
4	`view` state + zoom slider + `transform: scale()`	Zoom works (from the center)
5	Pointer drag + `clampOffset`	Pan works, never shows empty space
6	`zoomView` re-clamps the offset	Zoom out after panning stays covered
7	`getCropRect` + `cropImage` → Blob → preview	Save shows the correct cropped avatar
8	Mock `uploadAvatar` + status + Retry + stale-response guard	Full happy and failure paths
9	Circle mask, drag and drop zone, Cancel	Feels like a real product
10	Keyboard, wheel (non-passive), focus ring, `aria-live`, `role="alert"`	Accessible and comfortable
11	Revoke object URLs, image `onError`, same-file re-pick	No leaks, corrupt files handled

9. Cover all states and edge cases

State / edge case	What the user sees	How it is handled
Initial	Empty grey circle, "No avatar yet", drop zone	`source = null`, `status = 'idle'`
Dragging a file over	Drop zone highlighted	`dragging` state, `preventDefault` on dragover
Invalid type	"Please choose a JPG, PNG or WebP image."	`validateFile` before creating a URL
Too large	"Image must be under 5 MB."	`validateFile`
Image loading	"Loading image..." in the window, Save + slider disabled	`img = null` until `onLoad`
Corrupt / fake image	"Could not read this image." back to drop zone	`<img onError>`
Editing	Image in the window, circular guide, zoom slider	`view` state

State / edge case	What the user sees	How it is handled
Very wide / very tall image	Short side fills the window; pan only along the long side	cover scale + per-axis clamp
Small image (e.g. 100 x 100)	Upscaled to fill; output may look soft	accepted; could warn below a min size
Zoom at limits	Slider and keys stop at 1x and 3x	`clamp` in `zoomView`
Exporting	Button "Saving...", buttons disabled	`saving` state
Export fails	Error message, back to drop zone	`toBlob` null → reject → `onError`
Uploading	Dimmed preview, "Uploading..." (announced)	`status = 'uploading'`, `aria-live`
Upload success	"Avatar saved"	`status = 'success'`
Upload failure	"Upload failed." + Retry	`status = 'error'`, blob kept in `avatar`
New upload while old one pending	Only the newest result shown	request id in `requestRef`
Same file picked twice	Cropper opens again	reset `input.value = ''`
Cancel	Back to drop zone, avatar unchanged	`setSource(null)`
Wheel over the window	Zooms, page does not scroll	non-passive native listener
Touch device	Drag pans the image, not the page	pointer events + `touch-action: none`

10. Final structure

```
_34_avtar_upload_and_crop/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx                    # flow: source → crop → upload
    ├── App.css
    ├── components/
    │   ├── FileDropzone.jsx       # input + drag and drop
    │   ├── AvatarCropper.jsx      # pan / zoom / export
    │   └── AvatarPreview.jsx      # circle + upload status + retry
    ├── utils/
    │   ├── cropMath.js            # pure math (testable without DOM)
    │   └── image.js               # validateFile, cropImage (canvas → Blob)
    └── services/
        └── avatarApi.js           # uploadAvatar (mock)
```

11. Full working code

`src/utils/cropMath.js`

```
export const VIEWPORT = 280; // crop window size in CSS px
export const OUTPUT = 256; // exported avatar size in px
export const MAX_ZOOM = 3;

export const clamp = (n, min, max) => Math.min(max, Math.max(min, n));

// Smallest scale at which the image still covers the whole viewport ("cover").
export const getMinScale = (img, v = VIEWPORT) => Math.max(v / img.w, v / img.h);

// x / y = image center relative to viewport center, in CSS px.
// Clamp so the image edge never enters the viewport (no empty corners).
export function clampOffset(x, y, img, scale, v = VIEWPORT) {
  const maxX = (img.w * scale - v) / 2;
  const maxY = (img.h * scale - v) / 2;
  return { x: clamp(x, -maxX, maxX), y: clamp(y, -maxY, maxY) };
}

export function panView(view, x, y, img, v = VIEWPORT) {
  return { ...view, ...clampOffset(x, y, img, getMinScale(img, v) * view.zoom, v) };
}

// Zoom around the viewport center: scale the offset with the image, then re-clamp.
export function zoomView(view, nextZoom, img, v = VIEWPORT) {
  const zoom = clamp(nextZoom, 1, MAX_ZOOM);
  const ratio = zoom / view.zoom;
  return { zoom, ...clampOffset(view.x * ratio, view.y * ratio, img, getMinScale(img, v) * zoom, v) };
}

// Map the visible viewport square back to natural image pixels for canvas.drawImage.
export function getCropRect(view, img, v = VIEWPORT) {
  const scale = getMinScale(img, v) * view.zoom;
  const size = v / scale;
  return {
    sx: img.w / 2 - size / 2 - view.x / scale,
    sy: img.h / 2 - size / 2 - view.y / scale,
    size,
  };
}
```

`src/utils/image.js`

```
import { OUTPUT } from './cropMath';

export const ACCEPTED_TYPES = ['image/jpeg', 'image/png', 'image/webp'];
export const MAX_SIZE_MB = 5;

export function validateFile(file) {
  if (!file) return 'No file selected.';
  if (!ACCEPTED_TYPES.includes(file.type)) return 'Please choose a JPG, PNG or WebP image.';
  if (file.size > MAX_SIZE_MB * 1024 * 1024) return `Image must be under ${MAX_SIZE_MB} MB.`;
  return null;
}
```

```
// Draw only the cropped square into a small canvas and export it as a Blob.
export function cropImage(imgEl, rect, output = OUTPUT) {
  const canvas = document.createElement('canvas');
  canvas.width = canvas.height = output;
  const ctx = canvas.getContext('2d');
  ctx.imageSmoothingQuality = 'high';
  ctx.drawImage(imgEl, rect.sx, rect.sy, rect.size, rect.size, 0, 0, output, output);
  // Browsers that can't encode WebP fall back to PNG automatically.
  return new Promise((resolve, reject) =>
    canvas.toBlob((blob) => (blob ? resolve(blob) : reject(new Error('Could not export the image.'))),
'image/webp', 0.9),
  );
}
```

src/services/avatarApi.js

```
export const FAIL_RATE = 0.2;

// Mock upload. Real version:
// const body = new FormData(); body.append('avatar', blob, 'avatar.webp');
// const res = await fetch('/api/me/avatar', { method: 'POST', body });
// if (!res.ok) throw new Error('Upload failed'); return res.json();
export function uploadAvatar(blob) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() < FAIL_RATE) reject(new Error('Upload failed. Please retry.'));
      else resolve({ id: crypto.randomUUID(), size: blob.size, uploadedAt: Date.now() });
    }, 1000);
  });
}
```

src/components/FileDropzone.jsx

```
import { useState } from 'react';
import { ACCEPTED_TYPES, MAX_SIZE_MB } from '../utils/image';

export default function FileDropzone({ onFile }) {
  const [dragging, setDragging] = useState(false);

  const handleDrop = (e) => {
    e.preventDefault(); // stop the browser from opening the file
    setDragging(false);
    onFile(e.dataTransfer.files[0]);
  };

  return (
    <label
      className={`dropzone ${dragging ? 'dropzone--active' : ''}`}
      onDragOver={(e) => {
        e.preventDefault();
        setDragging(true);
      }}
      onDragLeave={() => setDragging(false)}
      onDrop={handleDrop}
    >
```

```
      <input
        type="file"
        accept={ACCEPTED_TYPES.join(',')}
        className="visually-hidden"
        onChange={(e) => {
          onFile(e.target.files[0]);
          e.target.value = ''; // allow picking the same file again
        }}
      />
      <span>
        Drag an image here or <u>browse</u>
      </span>
      <small>JPG, PNG or WebP · up to {MAX_SIZE_MB} MB</small>
    </label>
  );
}
```

`src/components/AvatarCropper.jsx`

```
import { useEffect, useRef, useState } from 'react';
import { MAX_ZOOM, VIEWPORT, getCropRect, getMinScale, panView, zoomView } from '../utils/cropMath';
import { cropImage } from '../utils/image';

const STEP = 10; // px per arrow key press
const KEY_MOVES = { ArrowLeft: [-STEP, 0], ArrowRight: [STEP, 0], ArrowUp: [0, -STEP], ArrowDown: [0,
STEP] };

export default function AvatarCropper({ src, onCancel, onCrop, onError }) {
  const imgRef = useRef(null);
  const viewportRef = useRef(null);
  const dragRef = useRef(null); // pointer start, kept out of state: no re-render needed
  const [img, setImg] = useState(null); // natural size { w, h } once loaded
  const [view, setView] = useState({ zoom: 1, x: 0, y: 0 });
  const [saving, setSaving] = useState(false);

  // React's onWheel is passive, so preventDefault (stop page scroll) needs a native listener.
  useEffect(() => {
    const el = viewportRef.current;
    if (!img) return;
    const onWheel = (e) => {
      e.preventDefault();
      setView((v) => zoomView(v, v.zoom - e.deltaY * 0.002, img));
    };
    el.addEventListener('wheel', onWheel, { passive: false });
    return () => el.removeEventListener('wheel', onWheel);
  }, [img]);

  const handlePointerDown = (e) => {
    if (!img) return;
    e.currentTarget.setPointerCapture(e.pointerId);
    dragRef.current = { px: e.clientX, py: e.clientY, x: view.x, y: view.y };
  };

  const handlePointerMove = (e) => {
    const d = dragRef.current;
    if (!d) return;
    setView((v) => panView(v, d.x + e.clientX - d.px, d.y + e.clientY - d.py, img));
  };
```

```
  const handleKeyDown = (e) => {
    if (!img) return;
    if (KEY_MOVES[e.key]) {
      e.preventDefault();
      const [dx, dy] = KEY_MOVES[e.key];
      setView((v) => panView(v, v.x + dx, v.y + dy, img));
    } else if (e.key === '+' || e.key === '=') {
      setView((v) => zoomView(v, v.zoom + 0.1, img));
    } else if (e.key === '-') {
      setView((v) => zoomView(v, v.zoom - 0.1, img));
    }
  };

  const handleSave = async () => {
    setSaving(true);
    try {
      onCrop(await cropImage(imgRef.current, getCropRect(view, img)));
    } catch (err) {
      onError(err.message);
      setSaving(false);
    }
  };

  const base = img && getMinScale(img); // zoom 1 = image just covers the viewport

  return (
    <div className="cropper">
      <div
        ref={viewportRef}
        className="cropper__viewport"
        style={{ width: VIEWPORT, height: VIEWPORT }}
        tabIndex={0}
        role="group"
        aria-label="Crop area. Drag or use arrow keys to move, plus and minus to zoom."
        onPointerDown={handlePointerDown}
        onPointerMove={handlePointerMove}
        onPointerUp={() => (dragRef.current = null)}
        onPointerCancel={() => (dragRef.current = null)}
        onKeyDown={handleKeyDown}
      >
        <img
          ref={imgRef}
          src={src}
          alt=""
          draggable={false}
          onLoad={(e) => setImg({ w: e.currentTarget.naturalWidth, h: e.currentTarget.naturalHeight
})}
          onError={() => onError('Could not read this image. Try another file.')}
          style={
            img
              ? {
                  width: img.w * base,
                  height: img.h * base,
                  transform: `translate(calc(-50% + ${view.x}px), calc(-50% + ${view.y}px))
scale(${view.zoom})`,
                }
              : { visibility: 'hidden' }
          }
        />
```

```
        {!img && <span className="cropper__loading">Loading image…</span>}
        <div className="cropper__mask" aria-hidden="true" />
      </div>

      <label className="cropper__zoom">
        Zoom
        <input
          type="range"
          min={1}
          max={MAX_ZOOM}
          step={0.01}
          value={view.zoom}
          disabled={!img}
          onChange={(e) => setView((v) => zoomView(v, Number(e.target.value), img))}
        />
      </label>

      <div className="cropper__actions">
        <button type="button" onClick={onCancel} disabled={saving}>
          Cancel
        </button>
        <button type="button" className="primary" onClick={handleSave} disabled={!img || saving}>
          {saving ? 'Saving…' : 'Save avatar'}
        </button>
      </div>
    </div>
  );
}
```

src/components/AvatarPreview.jsx

```
const LABELS = {
  idle: 'No avatar yet',
  uploading: 'Uploading…',
  success: 'Avatar saved',
  error: 'Upload failed.',
};

export default function AvatarPreview({ url, status, onRetry }) {
  return (
    <div className="preview">
      {url ? (
        <img src={url} alt="Your avatar" className={`preview__img ${status === 'uploading' ? 'is-busy'
: ''}`} />
      ) : (
        <div className="preview__img preview__img--empty" aria-hidden="true" />
      )}
      <p className={`status status--${status}`} aria-live="polite">
        {LABELS[status]}{' '}
        {status === 'error' && (
          <button type="button" onClick={onRetry}>
            Retry
          </button>
        )}
      </p>
    </div>
```

```
  );
}
```

`src/App.jsx`

```
import { useEffect, useRef, useState } from 'react';
import FileDropzone from './components/FileDropzone';
import AvatarCropper from './components/AvatarCropper';
import AvatarPreview from './components/AvatarPreview';
import { validateFile } from './utils/image';
import { uploadAvatar } from './services/avatarApi';
import './App.css';

export default function App() {
  const [source, setSource] = useState(null); // object URL of the picked file
  const [avatar, setAvatar] = useState(null); // { blob, url } of the cropped result
  const [status, setStatus] = useState('idle'); // idle | uploading | success | error
  const [error, setError] = useState('');
  const requestRef = useRef(0); // latest upload id, ignores stale responses

  // Object URLs hold the whole file in memory until revoked.
  useEffect(() => () => source && URL.revokeObjectURL(source), [source]);
  useEffect(() => () => avatar && URL.revokeObjectURL(avatar.url), [avatar]);

  const handleFile = (file) => {
    const err = validateFile(file);
    setError(err ?? '');
    if (!err) setSource(URL.createObjectURL(file));
  };

  const upload = async (blob) => {
    const id = ++requestRef.current;
    setStatus('uploading');
    try {
      await uploadAvatar(blob);
      if (id === requestRef.current) setStatus('success');
    } catch {
      if (id === requestRef.current) setStatus('error');
    }
  };

  const handleCrop = (blob) => {
    setAvatar({ blob, url: URL.createObjectURL(blob) }); // optimistic preview
    setSource(null);
    upload(blob);
  };

  const handleCropError = (message) => {
    setError(message);
    setSource(null);
  };

  return (
    <main className="app">
      <h1>Profile picture</h1>
      <AvatarPreview url={avatar?.url} status={status} onRetry={() => upload(avatar?.blob)} />
      {source ? (
        // key resets zoom/pan state whenever a new image is picked
```

```
        <AvatarCropper
          key={source}
          src={source}
          onCancel={() => setSource(null)}
          onCrop={handleCrop}
          onError={handleCropError}
        />
      ) : (
        <FileDropzone onFile={handleFile} />
      )}
      {error && (
        <p role="alert" className="error">
          {error}
        </p>
      )}
    </main>
  );
}
```

`src/App.css`

```
.app {
  max-width: 420px;
  margin: 0 auto;
  padding: 32px 16px;
  display: grid;
  gap: 20px;
  justify-items: center;
}

.preview { display: grid; justify-items: center; gap: 8px; }
.preview__img { width: 96px; height: 96px; border-radius: 50%; object-fit: cover; background: #e5e7eb;
}
.preview__img.is-busy { opacity: 0.5; }
.status { margin: 0; font-size: 14px; color: #6b7280; }
.status--success { color: #15803d; }
.status--error { color: #b91c1c; }

.dropzone {
  width: 100%;
  display: grid;
  gap: 4px;
  justify-items: center;
  padding: 40px 16px;
  border: 2px dashed #9ca3af;
  border-radius: 12px;
  cursor: pointer;
}
.dropzone--active, .dropzone:hover { border-color: #2563eb; background: #eff6ff; }
.dropzone:focus-within { outline: 3px solid #2563eb; outline-offset: 2px; }
.dropzone small { color: #6b7280; }

.cropper { display: grid; gap: 16px; justify-items: center; }
.cropper__viewport {
  position: relative;
  overflow: hidden;
  background: #111827;
  cursor: grab;
```

```
  touch-action: none; /* let pointer events pan instead of scrolling the page */
  user-select: none;
}
.cropper__viewport:active { cursor: grabbing; }
.cropper__viewport:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }
.cropper__viewport img { position: absolute; top: 50%; left: 50%; max-width: none; pointer-events:
none; }
.cropper__mask {
  position: absolute;
  inset: 0;
  border-radius: 50%;
  box-shadow: 0 0 0 9999px rgb(0 0 0 / 0.5); /* darken everything outside the circle */
  pointer-events: none;
}
.cropper__loading { position: absolute; inset: 0; display: grid; place-items: center; color: #fff; }
.cropper__zoom { display: flex; gap: 12px; align-items: center; width: 100%; }
.cropper__zoom input { flex: 1; }
.cropper__actions { display: flex; gap: 12px; }

button { padding: 8px 16px; border-radius: 8px; border: 1px solid #d1d5db; background: #fff; cursor:
pointer; font: inherit; }
button.primary { background: #2563eb; border-color: #2563eb; color: #fff; }
button:disabled { opacity: 0.6; cursor: not-allowed; }
.error { margin: 0; color: #b91c1c; }

.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
  white-space: nowrap;
}
```

`src/index.css`

```
*, *::before, *::after { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background: #f9fafb;
}
```

`src/main.jsx` stays as generated by Vite (renders `<App />` in `StrictMode`).

> **React Compiler gotcha:** this project has the React Compiler on. It memoizes the inline `onRetry={() => upload(avatar.blob)}` and reads `avatar.blob` **during render** to build its cache key, which crashes while `avatar` is `null`. Writing `avatar?.blob` fixes it. Good to mention if the interviewer asks about the compiler.

---

12. Wrap-up

Edge cases handled

- Wrong type, too large, corrupt file, export failure, upload failure with retry.
- Wide, tall and tiny images always cover the crop window; zoom out after panning stays covered.
- Stale upload responses ignored; object URLs revoked; same file can be re-picked.
- Mouse, touch, wheel and keyboard all drive the same pure `panView` / `zoomView`.

Trade-offs to mention

Decision	Why	Cost
Hand-written crop vs `react-easy-crop`	Shows understanding; zero dependency	Fewer features (rotation, pinch)
CSS transform preview, canvas only on Save	Smooth 60 fps, no per-frame canvas work	Two coordinate systems to keep in sync (solved by `getCropRect`)
Square output, circle by CSS	Works everywhere the avatar is shown (square or round)	Corners stored but hidden
256 px WebP output	Small upload (~10-30 KB)	Can't re-crop at high resolution later
Optimistic preview	Feels instant	Must handle failure (Retry)
Object URL vs `FileReader.readAsDataURL`	No base64 copy (33 % bigger), instant	Must revoke manually

What to add with more time

- **Tests:** unit tests for `cropMath` (already pure), RTL tests for validation and the upload states, a Playwright test that drags and checks the exported pixels.
- **Features:** pinch-to-zoom (two pointers), rotate 90°, zoom towards the cursor, minimum dimension check, show the original avatar until the new one is saved.
- **Performance:** decode huge images with `createImageBitmap` / downscale in a Web Worker with `OffscreenCanvas`; throttle pointer moves with `requestAnimationFrame` if needed.
- **Upload:** real `fetch` with `FormData`, progress via `XMLHttpRequest.upload.onprogress`, `AbortController` to cancel, presigned S3 URL, retry with backoff.
- **UX:** undo to the previous avatar, a "Reset" button, a dark theme.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why not crop on the server instead?

You'd upload the full 5 MB original instead of a 20 KB avatar, and the server would need an image library. Cropping on the client is faster for the user and cheaper for us. The server should still **re-validate and re-**

encode the image (never trust the client), but it doesn't need the crop logic.

Q2. Why validate on the client if the server validates too?

Client validation is for **fast feedback** (no wasted upload). Server validation is for **security**. `file.type` comes from the extension and can be faked, so the server checks the real bytes (magic numbers) and re-encodes.

Q3. What if the user uploads a 50 MB, 12000 x 9000 photo?

It's rejected by the size limit. If we allowed it, decoding would take ~430 MB of RAM and can crash mobile tabs. I'd decode with `createImageBitmap(file, { resizeWidth })` or downscale in a Worker with `OffscreenCanvas` before showing it.

State design

Q4. Why store `{ zoom, x, y }` in one object instead of three `useState`s?

Zooming changes the offset, so zoom and offset must update together. With one object and a functional update (`setView(v => zoomView(v, ...))`), every wheel event builds on the latest value. With three states, a handler would read stale `zoom` from its closure during fast wheel scrolling.

Q5. Why not store the crop rectangle or the scale in state?

They are **derived** from `img` + `view`. Storing them creates a second source of truth that can drift. Computing them is a few multiplications per render.

Q6. Why use `key={source}` on the cropper?

A new image must start at zoom 1, centered. Changing the `key` remounts the component, so all its state resets for free, with no "reset" effect that could run one render late.

Q7. Why is the drag start in a ref and not in state?

It never changes what's on screen; it's only read in the next `pointermove`. Putting it in state would cause an extra re-render on every pointer down for nothing.

Challenge: crop math

Q8. Walk me through `getCropRect`. How do you know it's right?

The window is `V` CSS px; at scale `s` that's `V / s` natural px. The image center is at `w/2`; the window center is `x / s` natural px away from it. So `sx = w/2 - x/s - size/2`. I check it with asserts: centered at zoom 1 on a 1000 x 500 image gives `{ sx: 250, sy: 0, size: 500 }`, and at the max right offset `sx = 0`.

Q9. What if the photo is rotated (EXIF orientation from a phone)?

Modern browsers apply EXIF orientation to both `<img>` and `drawImage` by default (`image-orientation: from-image`), so the preview and the export match. For old browsers I'd read the EXIF tag and rotate the canvas, or use `createImageBitmap` with `imageOrientation: 'from-image'`.

Q10. Why does zoom happen around the center, and how would you zoom towards the cursor?

Center zoom is just "multiply offset by the zoom ratio". To zoom towards the cursor, take the cursor position `p` relative to the window center and use `newOffset = p - (p - offset) * ratio`, then clamp. Same function, one more input.

Challenge: interaction

Q11. Why pointer events and not mouse + touch events?

One code path for mouse, touch and pen. `setPointerCapture` keeps sending moves even when the finger leaves the box, so the drag never gets "stuck". `touch-action: none` stops the browser from scrolling the page on touch.

Q12. Why not use React's `onWheel`?

React attaches wheel listeners as **passive**, so `preventDefault()` is ignored and the page scrolls while you zoom. A native `addEventListener('wheel', fn, { passive: false })` in an effect fixes it, and the cleanup removes it.

Q13. How would you add pinch-to-zoom?

Track active pointers in a `Map(pointerId → position)`. With two pointers, compute the distance between them; `zoom = startZoom * (distance / startDistance)` through `zoomView`. With one pointer, keep panning.

Q14. Is it accessible?

The crop window is focusable with a visible focus ring and a label explaining the keys; arrows pan, `+` / `-` zoom. The slider is a native labelled range input. Errors use `role="alert"`, upload status uses `aria-live="polite"`, and the file input stays in the tab order (visually hidden, not `display: none`).

Implementation steps

Q15. Why `URL.createObjectURL` and not `FileReader.readAsDataURL`?

An object URL is instant and points at the file in memory. A data URL is a base64 string ~33 % larger, created asynchronously, and copied into the DOM. The cost: we must call `revokeObjectURL`, which the effect cleanups do.

Q16. Why do you reset `e.target.value = ''` after picking?

If the user picks the same file again (e.g. after Cancel), the input value doesn't change, so `onChange` doesn't fire. Clearing it makes every pick fire.

Q17. Why `canvas.toBlob` and not `toDataURL`?

`toBlob` is async (doesn't block the main thread while encoding) and gives binary data ready for `FormData`. `toDataURL` is sync and base64, ~33 % bigger.

Q18. What if the image comes from another domain (e.g. the current avatar URL)?

Drawing a cross-origin image taints the canvas and `toBlob` throws a `SecurityError`. The server must send CORS headers and the `<img>` needs `crossOrigin="anonymous"`. Our flow only uses local files, so it's safe.

Upload and network

Q19. What if the API is slow, or the user picks a new image during the upload?

The status shows "Uploading..." and the preview is dimmed. Each upload gets an increasing id; only the latest one may set the status, so a slow old response can't overwrite a newer one. With a real `fetch` I'd also `abort()` the previous request with an `AbortController`.

Q20. How would you show upload progress?

`fetch` has no upload progress; use `XMLHttpRequest` and `xhr.upload.onprogress` (or chunked uploads). Store `progress` in state and render a progress bar. For a 20 KB avatar it's usually not worth it.

Q21. Why an optimistic preview? What if the upload keeps failing?

It feels instant. On failure we keep the blob, show "Upload failed" + Retry. A stricter version keeps showing the **old** avatar until the server confirms, and shows the new one only on success.

Performance and trade-offs

Q22. Why not redraw a canvas on every drag instead of using CSS transform?

Redrawing a large image 60 times a second is expensive. A CSS `transform` is handled by the compositor on the GPU with no layout. We touch canvas exactly once, on Save.

Q23. Why not use `react-easy-crop` or `cropperjs`?

In a real product I would, since they already handle pinch, rotation and edge cases. In an interview the point is to show I understand the math and events. The pure `cropMath` functions would also make migrating easy.

Q24. What if 1000 users change their avatar at the same moment?

The client part doesn't change; it's all local. On the backend: upload straight to object storage with **presigned URLs** (no app-server bandwidth), process asynchronously, serve through a CDN with a versioned URL (`avatar.webp?v=uploadedAt`) so caches update.

Q25. Why a 256 px output? What about retina screens?

A 128 px avatar on a 2x screen needs 256 px, so 256 covers most uses. If the app shows big profile headers, I'd export 512 px, or upload the full crop and let the server generate sizes.